



杭州电子科技大学
HANGZHOU DIANZI UNIVERSITY

《数据结构课程设计》

实验报告

实验 1：线性表的实现与操作

姓 名： xx

学 号： xx

专 业： 网络空间安全

实验时间： 2024.10.24-2024.11.7

指导老师： xx

杭州电子科技大学
网络空间安全学院

一、实验目的

- 1.掌握线性表顺序存储结构的表示与基本操作的实现
- 2.掌握线性表链式存储结构的表示与基本操作的实现
- 3.掌握栈与队列的操作与应用
- 4.掌握串的操作与应用

二、实验内容与实验结果

实验 1-1

1. 实验内容

I. 编写函数：

根据定义的顺序表结构，编写下列基本操作函数的 C/C++源代码：

(1) InitList(Sqlist &L)——利用 new 函数申请存储空间，构造一个空的顺序表表 L，L 的初始大小为 MAXSIZE。

(2) ClearList(Sqlist &L)——将顺序表的数据清空。只需将 L.length 清零，即 L.length=0 即可。

(3) int Listlength(Sqlist L)——返回 L 中数据元素个数。直接返回 L.length 的值即可。

(4) GetElem(Sqlist L, int i, ElemType &e)——用 e 返回 L 中第 i 个数据元素的值， $1 \leq i \leq L.Length$ 。请注意函数参数 i 与数组下标之间的关系（以下同）。

(5) int LocateList(Sqlist L, ElemType e)——返回 L 中第 1 个与 e 相等的数据元素位序，若 e 不存在，则返回 0。

(6) ListInsert(Sqlist &L, int i, ElemType e)——在 L 中的第 i 个位置之前插入新的数据元素 e， $1 \leq i \leq L.length+1$ 。

(7) ListDelete(Sqlist &L, int i, ElemType &e)——删除 L 中的第 i 个数据元素，用 e 返回其值， $1 \leq i \leq L.length$ 。

(8) ListTraverse(Sqlist L)——利用 printf 函数依次输出 L 的每个数据元素的值。

注：以上函数除了表明函数类型外，其他的函数类型名均采用 Status（以下同）

II. 测试应用案例：

应用案例 1——线性表元素的集合运算

将顺序表 La 和 Lb 看成 2 个集合，利用上述基本操作函数，分别编写出实现集合并、交、差运算的 3 个函数源码：

(1) 并运算 $La=La \cup Lb$: Union(Sqlist &La, Sqlist Lb)

(2) 交运算 $La=La \cap Lb$: Intersection(Sqlist &La, Sqlist Lb)

(3) 差运算 $La=La - Lb$: Difference(Sqlist &La, Sqlist Lb)

应用案例 2——归并排序（两个有序表的合并）

设 La 和 Lb 中的元素是非递减有序的，编写一个归并排序，即 $Lc=La+Lb$ 的函数源码：

MergeList(Sqlist La, Sqlist Lb, Sqlist &Lc)

应用案例 3——去掉线性表中的重复多余的元素

设 Lc 是一个有重复元素的顺序表，编写一个去掉顺序表中重复多余元素的函数源码：

Purge(Sqlist &Lc)

编写一个主函数 main()，来检验上述应用函数是否正确，实现以下操作：

（1）数据准备。生成 2 个顺序表 LA、LB，其数据元素分别为：

LA= (2,8,27,39,66,77,89)

LB= (6,18,27,59,65,77,89,120,140)

顺序表生成后分别输出它们各元素的值。

（2）将顺序表 LA 和 LB 看成 2 个集合，分别调用集合并、交、差运算的 3 个函数，并在每个函数调用后输出 LA 的元素值，观察函数执行结果是否正确。

注意：由于函数调用后将导致 LA 的元素发生变化，因此在做下一个函数调用前需要恢复 LA 的元素。以下同

（3）因为 LA 和 LB 中的元素是非递减有序的，调用有序表合并的函数 MergeList(Sqlist La, Sqlist Lb, Sqlist &Lc)，并在函数调用后输出 LC 的元素值。

（4）MergeList 函数合并后的有序表 LC 存在重复多余的元素，调用 Purge(Sqlist &Lc)去掉重复多余的元素，并在函数调用后输出 LC 的元素值。

2. 源程序代码 (*.CPP)

（见附件 sqliist1.cpp）

2. 运行结果

```
LA: 2 8 27 39 66 77 89
LB: 6 18 27 59 65 77 89 120 140
LA ∪ LB: 2 8 27 39 66 77 89 6 18 59 65 120 140
LA ∩ LB: 27 77 89
LA - LB: 2 8 39 66
LC : 2 6 8 18 27 27 39 59 65 66 77 77 89 89 120 140
LC (去重后): 2 6 8 18 27 39 59 65 66 77 89 120 140

-----
Process exited after 0.02277 seconds with return value 1
请按任意键继续. . . |
```

实验 1-2

1. 实验内容

I. 单链表基本操作函数实现:

根据定义的单链表结构, 编写下列基本操作函数的 C/C++ 源代码:

(1) `InitList(LinkList &L)`——初始化单链表 L。利用 `malloc` 函数申请一个头结点, 构造一个空的单链表 L, 此时 L 指向头结点。

(2) `ClearList(LinkList &L)`——清空单链表。将 L 中所有数据结点空间释放。只剩下一个头结点, 和初始化操作后的结果一样。

(3) `int Listlength(LinkList L)`——求单链表的长度, 即返回 L 中数据结点个数。

(4) `GetElem(LinkList L, int i, ElemType &e)`——用 e 返回 L 中第 i 个数据结点的值, 其中 $1 \leq i \leq \text{Listlength}(L)$ 。

(5) `LNode *LocateList(LinkList L, ElemType e)`——返回 L 中第 1 个与 e 相等的数据结点的地址(指针), 若 e 不存在, 则返回 NULL。

(6) `ListInsert(LinkList &L, int i, ElemType e)`——在 L 中的第 i 个数据结点之前插入新的数据结点 e, $1 \leq i \leq \text{Listlength}(L)+1$ 。

(7) `ListDelete(LinkList &L, int i, ElemType &e)`——删除 L 中的第 i 个数据 j 结点, 用 e 返回删除结点的值, $1 \leq i \leq \text{Listlength}(L)$ 。

(8) `ListTraverse(LinkList L)`——利用 `printf` 函数依次输出 L 的每个数据结点的值。

注: 以上函数除了表明函数类型外, 其他的函数类型名均采用 `Status` (以下同)。

II. 应用案例的实现:

应用案例 1——线性表元素的集合运算

将单链表 La 和 Lb 看成 2 个集合, 利用上述基本操作函数, 分别编写出实现集合并、交、差运算的 3 个函数源码:

(1) 并运算 $La = La \cup Lb$: `Union(LinkList &La, LinkList Lb)`

(2) 交运算 $La = La \cap Lb$: `Intersection(LinkList &La, LinkList Lb)`

(3) 差运算 $La = La - Lb$: `Difference(LinkList &La, LinkList Lb)`

应用案例 2——归并排序 (两个有序表的合并)

设 La 和 Lb 中的元素是非递减有序的, 编写一个归并排序, 即 $Lc = La + Lb$ 的函数源码:

`MergeList(LinkList La, LinkList Lb, LinkList &Lc)`

应用案例 3——去掉线性表中的重复多余的元素

设 Lc 是一个有重复元素的顺序表, 编写一个去掉单链表 Lc 中重复多余元素的函数源码:

`Purge(LinkList &Lc)`

编写一个主函数 `main()`, 来检验上述应用函数是否正确, 实现以下操作:

(1) 数据准备。生成 2 个顺序表 LA、LB, 其数据元素分别为:

$LA = 2 \rightarrow 8 \rightarrow 27 \rightarrow 39 \rightarrow 66 \rightarrow 77 \rightarrow 89 \rightarrow \wedge$

LB=6→18→27→59→65→77→89→120→140→∧

单链表生成后，以上述格式输出各元素值。

(2) 将单链表 LA 和 LB 看成 2 个集合，分别调用集合并、交、差运算的 3 个函数，并在每个函数调用后输出 LA 的元素值，观察函数执行结果是否正确。

注意：由于函数调用后将导致 La 的元素发生变化，因此在做下一个函数调用前需要恢复 LA 的元素。以下同

(3) 因为 LA 和 LB 中的元素是非递减有序的，调用有序表合并的函数 MergeList(LA, LB, LC)，并在函数调用后输出 LC 的元素值。

(4) MergeList 函数合并后的有序表 LC 存在重复多余的元素，调用 Purge(LC)去掉重复多余的元素，并在函数调用后输出 LC 的元素值。

2. 源程序代码 (*.CPP)

(见附件 sqliist2.cpp)

3. 运行结果

```
LA: 2->8->27->39->66->77->89->∧
LB: 6->18->27->59->65->77->89->120->140->∧
LA ∪ LB: 2->8->27->39->66->77->89->6->18->59->65->120->140->∧
LA ∩ LB: 27->77->89->∧
LA - LB: 2->8->39->66->∧
LC : 2->6->8->18->27->27->39->59->65->66->77->77->89->89->120->140->∧
LC (去重后): 2->6->8->18->27->39->59->65->66->77->89->120->140->∧
```

实验 1-3

1. 实验内容

1. 顺序栈基本操作函数的实现:

根据定义的顺序栈结构，编写下列基本操作函数的 C/C++源代码:

(1) InitStack(SqStack &S)——利用 malloc 函数申请存储空间，构造一个空的顺序栈 S，S 的初始大小为 STACK_INIT_SIZE。

(2) ClearStack(SqStack &S)——将顺序栈数据清空。只需做 S.top=S.base 赋值操作即可。

(3) StackLength(SqStack S)——返回 L 中数据元素个数。直接返回 S.top-S.base 的值即可。

(4) GetTop(SqStack S, SElemType &e)——用 e 返回栈顶元素的值。

(5) Push(SqStack &S, SElemType e)——将数据 e 进栈。

(6) Pop(SqStack &S, SElemType &e)——将栈顶元素出栈，用 e 返回其值。

II. 循环队列基本操作函数的实现:

根据定义的循环队列结构, 编写下列基本操作函数的 C/C++源代码:

- (1) InitQueue(SqQueue &Q)——利用 malloc 函数申请存储空间, 构造一个空的循环队列 Q, Q 的大小为 MAXQSIZE。
- (2) ClearQueue(SqQueue &Q)——将循环队列数据清空。
- (3) QueueLength(SqQueue Q)——返回 Q 中数据元素个数。
- (4) EnQueue(SqQueue &Q, QElemType e)——数据 e 入队。
- (5) DeQueue(SqStack &Q, QElemType &e)——数据出队, 用 e 返回其值。
- (6) QueueEmpty(SqQueue Q)——判对空。
- (7) QueueFull(SqQueue Q)——判队满。

III. 应用案例

- (1) 利用顺序栈的基本操作函数, 编制一个将十进制整数 m 转换成 n 进制(n=2 或 8)整数的数制转换函数 Conversion(m,n) , 并在函数中输出转换结果。
- (2) 编一个函数 QueueReverse(Q), 利用栈将将队列中的元素逆序重排, 即队头元素变成队尾元素。

IV. 主函数 main()

- (1) 数制转换

输入一个十进制数 m, 调用 Conversion(m,n) 函数将 m 转换成二进制数和八进制数。

- (2) 循环队列操作

- <1>连续入队 9 个整数后, 显示队列 Q 的 front 和 rear 的值;
- <2>若再入队 1 个整数, 显示此时队列的状态信息;
- <3>连续出队 5 个整数, 再入队 2 个整数, 显示此时的队列 Q 的长度与 front 和 rear 的值;
- <4>调用函数 QueueReverse(Q), 将队列元素逆序重排, 然后将队列元素出队并输出。

2. 源程序代码 (*.CPP)

(见附件 stack and queue.cpp)

3. 运行结果

```
请输入一个十进制整数: 15
15 转换为 2 进制后结果为: 1111
15 转换为 8 进制后结果为: 17
连续入队9个整数后, 队列头序号: 0, 队列尾序号: 9
队列空间已被占满, 无法加入新元素
连续出队5个整数, 再入队2个整数后, 队列长度: 6, 队列头序号: 5, 队列尾序号: 1
队列元素逆序重排后:12 11 9 8 7 6
```

实验 1-4

1. 实验内容

I. 串基本操作函数的实现:

编写下列基于存储结构的基本操作函数的 C/C++ 源代码。

- (1) StrAssign(HString &S, chars)——将字符串常量 chars 赋给串变量 S;
- (2) StrCompare(HString S, HString T)——比较串 S 与 T 的值, 若 $S > T$, 则返回 1; 若 $S = T$, 则返回 0; 否则返回 -1。
- (3) StrLength(HString S)——返回串 S 的长度。
- (4) Concat(HString &S, HString T1, HString T2)——将串 T1 和 T2 连接在一起赋给串变量 S。
- (5) SubString(HString &sub, HString S, int pos, int len)——用 sub 返回串 S 的第 pos 个字符起长度为 len 的子串, $1 \leq pos \leq \text{StrLength}(S)$, $1 \leq len \leq \text{StrLength}(S) - pos + 1$
- (6) StrTraverse(HString S)——遍历 (输出) 字符串 S 中的各个字符。
- (7) StrCopy(HString &S, HString T)——将串 T 赋给串 S。

II. 串其他操作函数的实现:

调用上述串的基本操作函数, 编制下列串的操作函数。

- (1) Index(HString S, HString T, int pos)——返回串 T 在串 S 中第 pos 个字符开始出现的位置, 若串 T 不是串 S 的子串, 则返回 0。 $1 \leq pos \leq \text{StrLength}(S)$ 。利用 SubString() 和 StrCompare() 实现。
- (2) Replace(HString &S, HString T1, HString T2)——将 S 串中所有的子串 T1 用串 T2 代替。利用 SubString()、StrCompare() 和 Concat() 实现。
- (3) StrInsert (HString &S, int pos, HString T)——在串 S 的第 pos 个字符之前插入串 T。 $1 \leq pos \leq \text{StrLength}(S) + 1$ 。利用 SubString() 和 Concat() 实现。
- (4) StrDelete(HString &S, int pos, int len)——删除串 S 中从第 pos 个字符开始连续的 len 个字符。 $1 \leq pos \leq \text{StrLength}(S) - len + 1$, 利用 SubString() 和 Concat() 实现。

III. 编写一个主函数 main(), 检验上述操作函数是否正确, 实现以下操作:

- (1) 数据准备。利用 StrAssign() 将设定的字符串常量 (自行设定) 赋给 3 个字符串变量 T1, T2, T3, 然后利用 StrTraverse() 输出字符串字符。
- (2) 利用 StrCopy() 将串 T1 赋给串 S, 并输出 S 的字符。
- (3) 利用 StrInsert() 将串 T2 插入到 S 的尾部, 并输出 S 的字符。
- (4) 利用 StrReplace() 将串 S 中的 T1 替换成 T3, 并输出 S 的字符。
- (5) 利用 Index() 查找串 T2 在 S 中的位置, 并输出位置结果。
- (6) 利用 StrDelete() 删除 S 中子串 T2, 并输出删除后 S 的结果。

2. 源程序代码 (*.CPP)

(见附件 string.cpp)

3. 运行结果

```
T1: Hello
T2: World
T3: Everyone
将串T1赋给串S后: Hello
将串T2插入到串S尾部后: HelloWorld
串S中的T1替换成T3后: EveryoneWorld
串T2在S中的位置: 9
将串S删除字串T2后: Everyone
```

三、实验小结

在前两个实验中，我深入探讨了线性表的两种存储结构——顺序存储和链式存储。通过实现基本操作及集合运算，我不仅理解了它们的优缺点，也提升了自己的编码能力和逻辑思维。在顺序存储结构的实验中，我认识到数组的固定大小限制可能导致溢出，因此在插入新元素时需考虑空间的有效管理。实现动态扩容虽然增加了复杂性，但却使得程序更加灵活。我通过使用 `malloc` 分配新的内存空间，并在数据插入时调整数组的长度，成功解决了这个问题。而在链式存储结构的实验中，我发现链表在插入和删除操作上更加高效，但在内存管理方面则显得更加复杂。尤其是在清空链表时，必须小心处理每个节点的指针，确保没有内存泄露。这个过程让我深刻理解了动态内存分配的本质，以及如何在释放内存时保持数据的完整性。两次实验让我意识到算法不仅是一个理论概念，而是解决实际问题的工具。通过集合运算、归并排序等功能的实现，我看到了数据结构在实际应用中的重要性。无论是顺序表的静态结构还是链表的动态特性，它们都在不同场景下发挥着独特的作用。我也体会到，编程不仅仅是实现功能，更多的是解决问题和优化过程。每次遇到错误时，我都会仔细分析，查阅资料，最终找到解决方案。这一过程锻炼了我的耐心和逻辑推理能力，使我在面对复杂问题时更加从容。

在后两个实验中，我分别实现了顺序栈和循环队列的操作，以及基于堆分配存储的字符串操作。通过对顺序栈的基本操作函数编写，如入栈、出栈等，我深入理解了栈的 LIFO 特性及其在数据存储中的灵活应用。同时，通过循环队列的操作，我掌握了队列的 FIFO 特点，特别是循环存储结构如何避免“假满”问题。实验过程中，循环队列的指针操作加深了我对逻辑处理的理解，尤其是在判空、判满和动态调整队首、队尾指针时，进一步提升了编程技巧。

在串的操作实验中，我学会了基于堆分配存储的动态内存管理，通过编写字符串赋值、比较、插入、删除等功能，增强了对字符串操作的理解和 C/C++ 的指针使用能力。特别是通过函数组合实现字符串替换、查找等复杂操作，我体会到如何分解任务并运用已有的函数模块实现复杂逻辑的解决方案。整体上，这些实验让我在代码优化和数据结构应用上收获颇丰，掌握了更多的算法实现技巧。

总结来说，这几个实验让我在实践中加深了对数据结构的理解，提升了编码能力，也让我在解决问题的过程中收获了宝贵的经验和信心。